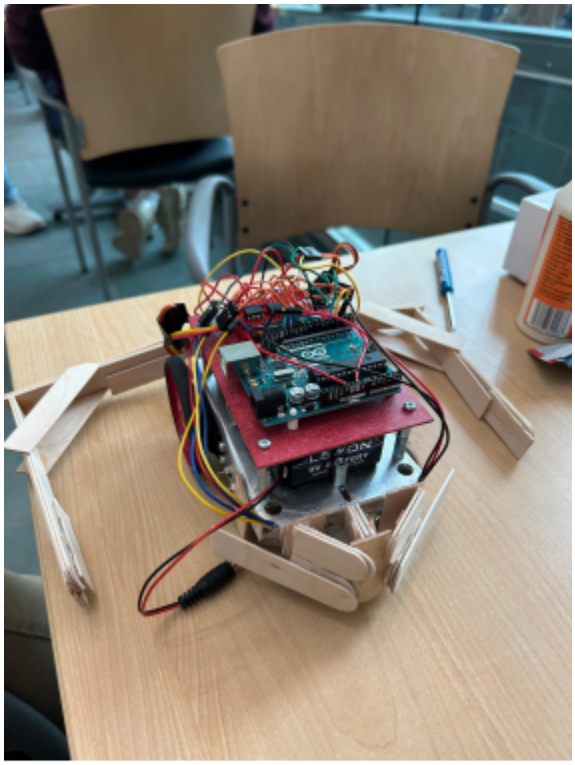
MAE 3780 Robot Project Final Report Overview

Group 53: Christopher Miranda Soto (cm2233), Hugo Larios (hdl45), Mathis Vigneau (mav222)

1. Robot Design and Strategy Overview

Our strategy for this competition was to deploy a grabbing mechanism with maximum reach to gather cubes, move efficiently within the range of where most cubes might be, and then shut off after the robot had a considerable amount of time to collect at least half of the cubes.

Our grabbing mechanism consists of two servos that rotate from an “up” position where the starting size requirement is met, to a “down” deployment mode, as shown in the image, which the robot stays in for the entirety of the round. We wanted our design to be lightweight, easily fixable, and easily adaptable, so we decided to use popsicle sticks since we can fashion them in several different ways. When deployed, the popsicle sticks cover the entirety of the max area possible and effectively hold all cubes that the robot runs into with the help of the popsicle stick ram in front, which redirects cubes directly in front of the robot.



This is Frank. He’s a little shy. Say “Hi Frank!”

Our robot’s electrical design involves an Arduino Uno which controls the robot, two servos to rotate arms, a 9V battery which powers the servos and wheel motors, two H-bridges to move the motors forward and backward, a color sensor, and two QTI sensors near each wheel to detect the black border. Refer to *Appendix B* for the detailed circuit diagram.

Our robot’s actions were controlled by Arduino code that we personally programmed according to our electrical components. We wanted our code to ensure that the robot could get moving as quickly as possible, follow an intelligent path that targets areas with the most blocks while being adaptable to obstacles like running into other robots, and correctly run all electrical components. Refer to *Appendix D* for a simplified flowchart of the algorithm and *Appendix E* for the code used in competition.

2. Design Process Reflection

At the beginning of the project, we all met to discuss various options for how we wanted to build the robot. We initially decided on creating a front-side harvesting system, inspired by combine harvesters used to cut and collect wheat. This harvester would be positioned such that it would collect cubes by pulling them in, and place them into a small opening for storage. After pitching this idea for the first milestone, we used provided wheels, servo motors, and sensors to provide our robot with mobility. This was sufficient for passing the next two milestones, which also required a lot of back-and-forth for coding. Our primary prototype and testing method included attaching the physical components, then running initial code. When something went wrong, we would check that everything was connected correctly, then check the code for any issues. Much of the building and programming of the robot was a collective effort, where any ongoing issue was looked at by all of us until it was fixed.

Our main roadblock in this project was a lack of sufficient materials. There was an early miscommunication in the team regarding our first parts order, which gave us the incorrect components for our robot. This continued again into the second parts order, which left us scrambling to pass our fourth milestone. However, we had a surplus of popsicle sticks, some L-brackets, and two provided servo motors, so we were able to come up with an alternative design for the robot. Here, we came up with our final design, which was to have the two servo motors on the back of the robot lower a set of arms. These arms would be lowered as the robot moved forward, allowing it to reach the center and collect cubes quickly. With this, we were able to complete the final milestone.

For the final improvements to the robot, we optimized the code so that the robot would gather cubes in the middle, then branch off and hide in a corner. We also added some popsicle sticks towards the front end of the robot, to prevent the possibility of any cubes getting stuck underneath. We troubleshooted until our robot was able to consistently gather more than half of the cubes on its own, then made backup components in case anything ended up breaking off during the competition.

3. Competition Analysis

During the competition, our robot performed below what we were hoping for. Our robot won two matches and lost 3 matches. The program executed well and overall achieved what we intended but we ran into unprecedented circumstances.

We quickly realized that our collection mechanism was very weak structurally and would break in every round it was used, as soon as it encountered the opponent's robot. We also encountered a strange problem in which our robot would start to spin out and then just stop. We knew the code was not the problem, but we could not figure out what was wrong in the circuit. We realized right after our second-to-last round that a wire came loose, which was the wire that allowed the QTI to communicate with the Arduino. This was a tragedy as we had lost three matches largely due to this wire that had completely interfered with our robot's functionality. After fixing this issue, our robot performed as intended and won the last round. This was a very unexpected issue we had, and looking back, had we looked more intently early on, we would have been able to fix it and likely win more matches.

The strongest point of our robot was its intelligence and how fast the code was being executed, as this allowed us to go to the center before our opponents and snag the cubes away from them. The weak points of the robot were definitely the weak wires and the fragility of the collecting mechanism. When broken, it would lose half of its cubes and thus cut our chances of winning substantially.

4. Conclusions

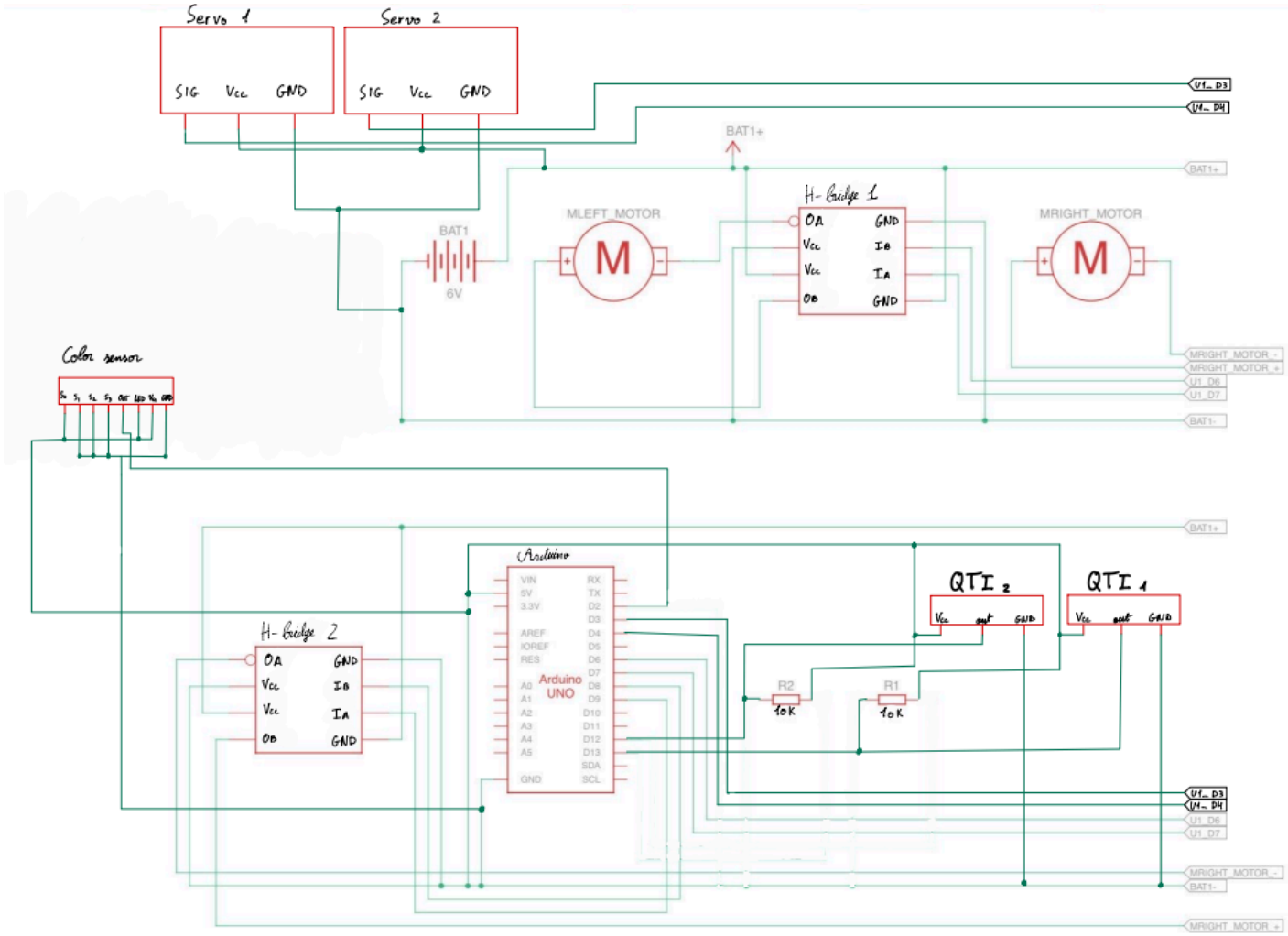
Overall, the robot mostly struggled with the fragility of its components. Wires could be unplugged and the arms could break off, which could have been avoided with stronger material or a better structural design. We believe that there are two possible solutions to this problem: we either needed to put in the additional time to make sure that the arms were secure and the wires were stuck in place, or we could have better coordinated our initial materials order so that we could have worked with our initial design. This would have made our robot more reliable, and thus have performed better during the competition.

Aside from the main difficulties we faced during competition, we could have also added various other components that would support our robot. Using larger wheels or utilizing a gear ratio would have made our robot move quicker. This would allow us to reach the center faster than our opponent, and thus collect more cubes. Additionally, we could have come up with a way to interfere with the enemy robot. One of our early ideas was to have a small catapult launch black wood chips at the opponent, in hopes that it would trick the robot into thinking it hit the boundary. This idea was scrapped, however, when our focus had to go into making our final design.

The performance of our robot, and the overall quality of its creation, was limited to the lack of necessary parts for our initial design. We had to improvise and come up with a final design with little time remaining, resulting in fragile components that would affect our robot's performance during competition. As advice for future students, we would say to be more proactive in the robot's design and construction. It feels as though there is more time that you actually have, and doing things early allows you to catch mistakes that take time to fix.

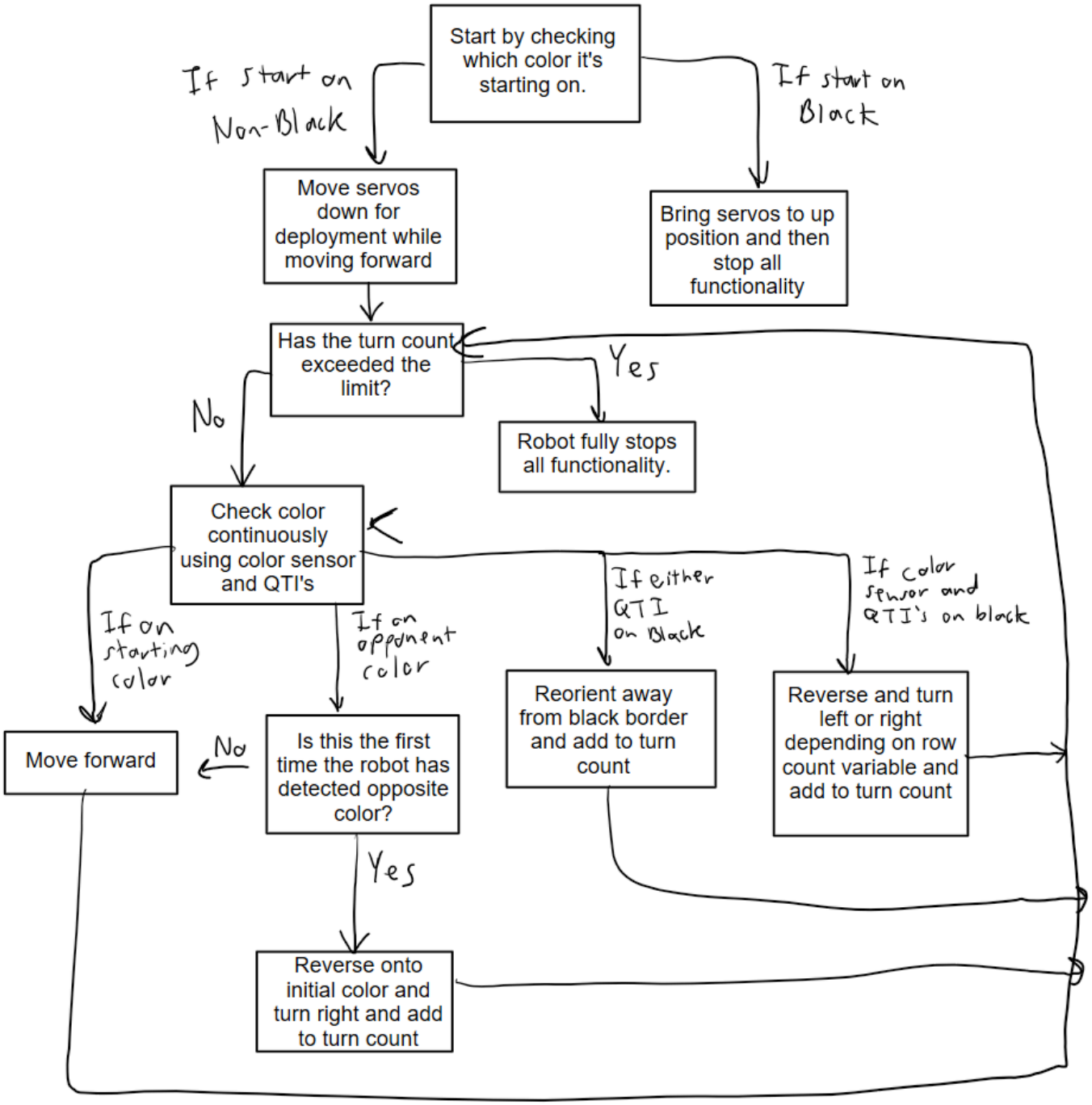
5. Appendix A:

6. Appendix B (Circuit Diagram):



7. Appendix C:

8. Appendix D (Algorithm Flowchart):



9. Appendix E (Robot Arduino Code):

```
//global variables
volatile unsigned int period = 0;
```

```
volatile unsigned int timer = 0;
volatile unsigned int currentcolor = 0;
volatile unsigned int initialcolor = 0;
volatile unsigned int current = 0;

ISR(PCINT2_vect) { //interrupt function to reset on rising edge
  if (PIND & (1 << 2)) {
    TCNT1 = 0;
  }
  else {
    timer = TCNT1;
  }
}

// Initialize interrupt and timer for color detection
void initColor() {
  //define input
  DDRD &= ~(1 << PD2);   // PD2 is 0 so input

  //initialize pin change interrupt
  PCICR |= (1 << PCIE2);
  PCMSK2 |= (1 << PCINT18);

  //initialize timer
  TCCR1A = 0;
  TCCR1B = (1 << CS10);
  TCNT1 = 0;

}



int getColor() { //get color function, returns 0 if black, 10 if yellow,
20 if blue
  PCMSK2 |= (1 << PD2);
  _delay_ms(10); //add short delay
  PCMSK2 &= ~(1 << PD2);//disable bit for interrupts until you call
function again

  // Timer is half the period, so full period = 2*timer each timer tick is
1/16 MHz = 0.0625 us,
```

```
  // so period in microseconds = 2 * timer * 0.0625 = timer / 8
  period = timer / 8;

  // Determine the color, and record into "period"
  if(period > 350) {
    period = 0;  // black
  }
  else if (period < 150) {
    period = 10;  //yellow
  }
  else {
    period = 20; // blue
  }
  return period; // Return "period"

}

int main(void){ // setup code that only runs once
  Serial.begin(9600); // start serial monitor for debugging
  sei(); // enable global interrupts
  initColor(); // call initColor function

  DDRB = 0b00000011; // Initialize right motor
  DDRD = 0b11011000; // Initialize left motor and left/right servos
  period = getColor(); // Assign "period" to the current color
  initialcolor = period; // Use "period" to determine the starting color
  current = period; // Use "period" to set the current color
  Serial.println(initialcolor); //check initial color

  // Main tracking variables for algorithm
  int rowCount = 0; //rowcount for knowing turn direction
  int servo = 0; //state of servo (up or deployed)
  int turncount = 0; //amount of turns before stopping
  int yell = 0; //variable for crossing color

  Serial.print("Current Color: "); //show current color for debugging
  if (period == 0) {
    Serial.println("Black");
  } // Print the color
  else if (period == 10) {
```

```
    Serial.println("Yellow");
  }
  else {
    Serial.println("Blue");
  }
  if (current == 0){ //reset servo code on black and dont move
    for (int i = 0; i < 50; i++) { //positional servo
      PORTD |= (1 << PD4);
      _delay_ms(2);
      PORTD &= ~(1 << PD4);
      _delay_ms(18);
      servo = 1;
    }
    //360 servo code for up position
    for (int i = 0; i < 20; i++) {   //integer determines how far servo
rotates

      PORTD |= (1 << PD3);
      _delay_ms(1);      // forward direction
      PORTD &= ~(1 << PD3);
      _delay_ms(19);
    }
    for (int i = 0; i < 20; i++) {
      PORTD |= (1 << PD3);
      _delay_ms(1.5);   // stop
      PORTD &= ~(1 << PD3);
      _delay_ms(18.5);
    }
  }
  else{ //down position for deployment
    drive_forward(); //move forward while dropping arms to save time
    //positional servo deploy code
    for (int i = 0; i < 70; i++) {
      PORTD |= (1 << PD4);
      _delay_ms(1);
      PORTD &= ~(1 << PD4);
      _delay_ms(19);
      servo = 0;
    }
    //360 servo deploy code
```

```
    for (int i = 0; i < 15; i++) {   //integer determines how far servo
rotates
      PORTD |= (1 << PD3);
      _delay_ms(2);      // forward direction
      PORTD &= ~(1 << PD3);
      _delay_ms(18);
    }
    // STOP PD3 (keep sending stop pulses)
    for (int i = 0; i < 20; i++) {
      PORTD |=(1 << PD3);
      _delay_ms(1.5);    // stop
      PORTD &= ~(1 << PD3);
      _delay_ms(18.5);
    }
    stop(20);
    drive_forward_delay(600); //immediately go to center (600 is 5.25in)
    while(1){ // code that loops forever
      current = getColor(); //check color
      //black border check first
      if (turncount <= 6){ // amount of turns before stopping
        if (current == 0 && (PINB & (1 << 5)) && (PINB & (1 << 4))) {
//all sensors on black
          Serial.println("TURN");
          stop(100);
          drive_backward_delay(200); //reverse back to color
          if (rowCount % 2 == 0) { //if even row turn right
            drive_right_delay(450);
            drive_forward_delay(200);
            drive_right_delay(450);
            turncount = turncount + 1;
          }
          else{ //if odd row turn left
            drive_left_delay(450);
            drive_forward_delay(600);
            drive_left_delay(450);
            turncount = turncount + 1;
          }
          rowCount = rowCount + 1;
          Serial.println("rowCount");
        }
```

```
        else if (PINB & (1 << 5)){ //left qti on black, reorient and add
turn count
          drive_right_delay(250);
          turncount = turncount + 1;
        }
        else if (PINB & (1 << 4)){ //right qti on black, reorient and add
turn count
          drive_left_delay(250);
          turncount = turncount + 1;
        }
        else if (current == initialcolor || yell==1){ //sweeping but other
color has been passed so now can go past it
          drive_forward();
          Serial.println("FORWARD");
        }
        else if (current != initialcolor && yell == 0){//first time hit
opponent color, go to side
          drive_backward_delay(200); //reverse back to color
          drive_right_delay(450); //drive right
          Serial.println("Reorient");
          turncount = turncount + 1;
          yell = yell +1; //yell was 0 now 1 for rest of round meaning can
cross color
        }
      }
      else{
        stop(50000); //after finishing turns, stop
      }
    }
  }
}

//movement functions

// option 1: movement functions with no delay
void drive_forward(){
  PORTB = 0b00000001; // Move "right" motor forward
  PORTD = 0b10000000; // Move "left" motor forward

  // Serial.println("I am moving forward");
```

```
}

void drive_backward(){
  PORTB = 0b00000010; // Move "right" motor backwards
  PORTD = 0b01000000; // Move "left" motor backwards

  // Serial.println("I am moving backwards");
}

void drive_right(){
  PORTB = 0b00000010; // Move "right" motor backwards
  PORTD = 0b10000000; // Move "left" motor forward

  // Serial.println("I am turning right");
}

void drive_left(){
  PORTB = 0b00000001; // Move "right" motor forward
  PORTD = 0b00000000; // Move "left" motor backwards

  // Serial.println("I am turning left");
}

// option 2: movement functions with built-in delay
void drive_forward_delay(int t){
  PORTB = 0b00000001; // Move "right" motor forward
  PORTD = 0b10000000; // Move "left" motor forward

  _delay_ms(t);     // wait for t milliseconds
}

void drive_backward_delay(int t){
  PORTB = 0b00000010; // Move "right" motor backwards
  PORTD = 0b01000000; // Move "left" motor backwards

  _delay_ms(t);    // wait for t milliseconds
}

void drive_right_delay(int t){
  PORTB = 0b00000010; // Move "right" motor backwards
```

```
  PORTD = 0b10000000; // Move "left" motor forward

  _delay_ms(t);   // wait for t milliseconds
}

void drive_left_delay(int t){
  PORTB = 0b00000001; // Move "right" motor forward
  PORTD = 0b01000000; // Move "left" motor backwards

  _delay_ms(t);   // wait for t milliseconds
}

// Stop the robot for a set amount of time
void stop(int t){
  PORTB = 0b00000011; // Stop "right" motor
  PORTD = 0b11000000; // Stop "left" motor

  _delay_ms(t);
}
```